

LAB # 12

Manipulation of inheritance and super keyword in java

Objective

To study inheritance and types of inheritance and using super to call superclass constructors.

Lab Goals

- Introduction to inheritance
- How **super** is used to call superclass constructors.
- Types of inheritance in Java.

Inheritance

Inheritance in Java is a mechanism where a new class inherits properties and behaviors (methods) from an existing class. The class that is inherited from is called the superclass or base class, and the class that inherits is called the subclass or derived class. This allows for code reusability and facilitates the creation of hierarchical relationships between classes.

Syntax demonstrated using an Example Code:

```
// Superclass
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}
// Subclass inheriting from Animal
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // Accessing eat() method from superclass Animal
        dog.bark(); // Accessing bark() method from subclass Dog
    }
}
```

Output:

```
Animal is eating
Dog is barking
```

Calling the superclass constructor using “super”:

Now, let's see how super is used to call superclass constructors. In Java, when a subclass is instantiated, the constructor of its superclass is called implicitly. However, if the superclass has parameterized constructors, and the subclass constructor needs to explicitly call one of them, **super()** is used.

Example Code:

```
// Superclass
class Animal {
    Animal() {
        System.out.println("Animal constructor called");
    }
}

// Subclass inheriting from Animal
class Dog extends Animal {
    Dog() {
        super(); // Calling superclass constructor explicitly
        System.out.println("Dog constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
    }
}
```

Output:

```
Animal constructor called
Dog constructor called
```

Types of inheritance in Java:

1. **Single Inheritance:** A subclass inherits from only one superclass.
2. **Multilevel Inheritance:** A subclass inherits from another subclass.
3. **Hierarchical Inheritance:** Multiple subclasses inherit from the same superclass.
4. **Multiple Inheritance (through interfaces):** A class implements multiple interfaces, effectively inheriting from all of them.

Example Code of Single Inheritance:

```
// Superclass
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

// Subclass inheriting from Animal
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // Inherited method
        dog.bark(); // Own method
    }
}
```

Output:

```
Animal is eating
Dog is barking
```

Example Code of Multilevel Inheritance:

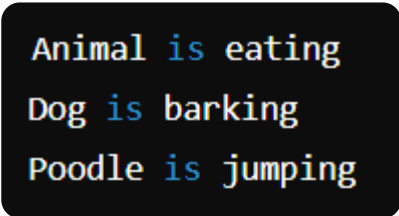
```
// Superclass
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

// Subclass inheriting from Animal
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

```
}

// Subclass inheriting from Dog
class Poodle extends Dog {
    void jump() {
        System.out.println("Poodle is jumping");
    }
}

public class Main {
    public static void main(String[] args) {
        Poodle poodle = new Poodle();
        poodle.eat(); // Inherited method from Animal
        poodle.bark(); // Inherited method from Dog
        poodle.jump(); // Own method
    }
}
```

Output:

```
Animal is eating
Dog is barking
Poodle is jumping
```

Example Code of Hierarchical Inheritance:

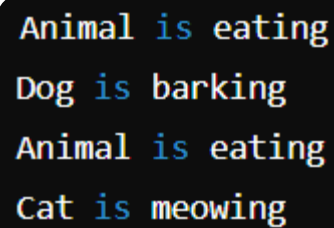
```
// Superclass
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

// Subclass inheriting from Animal
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

```
// Another subclass inheriting from Animal
class Cat extends Animal {
    void meow() {
        System.out.println("Cat is meowing");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // Inherited method from Animal
        dog.bark(); // Own method

        Cat cat = new Cat();
        cat.eat(); // Inherited method from Animal
        cat.meow(); // Own method
    }
}
```

Output:

```
Animal is eating
Dog is barking
Animal is eating
Cat is meowing
```

Example Code of Multiple Inheritance (through interfaces):

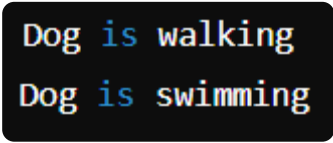
```
// Interface Walkable
interface Walkable {
    void walk();
}

// Interface Swimmable
interface Swimmable {
    void swim();
}

// Class implementing multiple interfaces
class Dog implements Walkable, Swimmable {
    public void walk() {
        System.out.println("Dog is walking");
    }
}
```

```
    public void swim() {
        System.out.println("Dog is swimming");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.walk(); // Implemented method from Walkable
        dog.swim(); // Implemented method from Swimmable
    }
}
```

Output:

```
Dog is walking
Dog is swimming
```

Exercise

- A. Create a class Card, and its children classes are Valentine, Holiday, and Birthday. A card class will have a greeting () method that writes out a greeting. Each type of card contains an appropriate greeting. The Holiday card says "Season's Greetings." The Birthday card says "Happy Birthday." The Valentine card says "Happy Valentine Day". Create objects of the three child classes: Valentine, Holiday, and Birthday and show the polymorphic behavior (call the greeting methods from each object).
- B. Develop a registration system for a University. It should consist of three classes namely Student, Teacher, and Course. For our example, a student needs to have a name, roll number, address and GPA to be eligible for registration. Therefore choose appropriate data types for encapsulating these properties in a Student object/s. Similarly a teacher needs to have name, address, telephone number, and a degree (or a list of degrees) he has received. Finally courses must have a name, students (5 students) registered for the course, and a teacher assigned to conduct the course. Create an object of Course with 5 Students and a Teacher. A call to a method, say printDetails(), of the selected course reference should print name of the course, details of the teacher assigned to that course, and names and roll numbers of the students enrolled with the course.
- C. Create a superclass Vehicle with a method start(). Then, create a subclass Car that extends Vehicle and adds a method drive(). Demonstrate single inheritance by creating an instance of Car and calling both methods.

Expected Output:

```
Vehicle is starting  
Car is driving
```

- D. Create a superclass Person with a method walk(). Create a subclass Employee that extends Person and adds a method work(). Then, create another subclass Manager that extends Employee and adds a method lead(). Demonstrate multilevel inheritance by creating an instance of Manager and calling all three methods.

Expected Output:

```
Person is walking  
Employee is working  
Manager is leading
```

- E. Create a superclass Device with a method powerOn(). Create two subclasses Phone and Laptop that both extend Device. Phone should add a method call(), and Laptop should add a method code(). Demonstrate hierarchical inheritance by creating instances of Phone and Laptop, and calling their respective methods along with the inherited method.

Expected Output:

```
Device is powered on  
Phone is calling  
Device is powered on  
Laptop is coding
```